

Steering decision

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing a right turn.
2. Learn the newly revealed key source code.

2. Right-click to access source code

Right-click source code path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

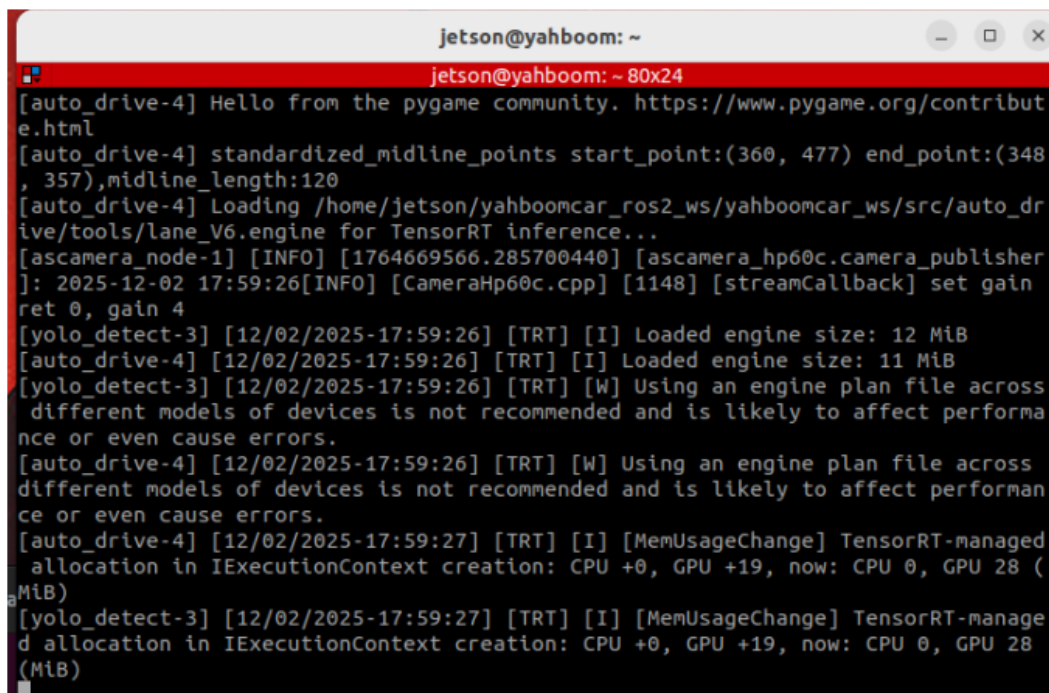
```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Start camera + chassis + road signs and lane lines

```
ros2 launch auto_drive auto_drive_bringup.launch.py
```

A terminal window titled 'jetson@yahboom: ~' with a red title bar. The terminal shows the output of the command 'ros2 launch auto_drive auto_drive_bringup.launch.py'. The output includes a welcome message from the pygame community, lane detection parameters, camera node initialization, and TensorRT engine loading for yolo_detect-3 and auto_drive-4. The terminal text is as follows:

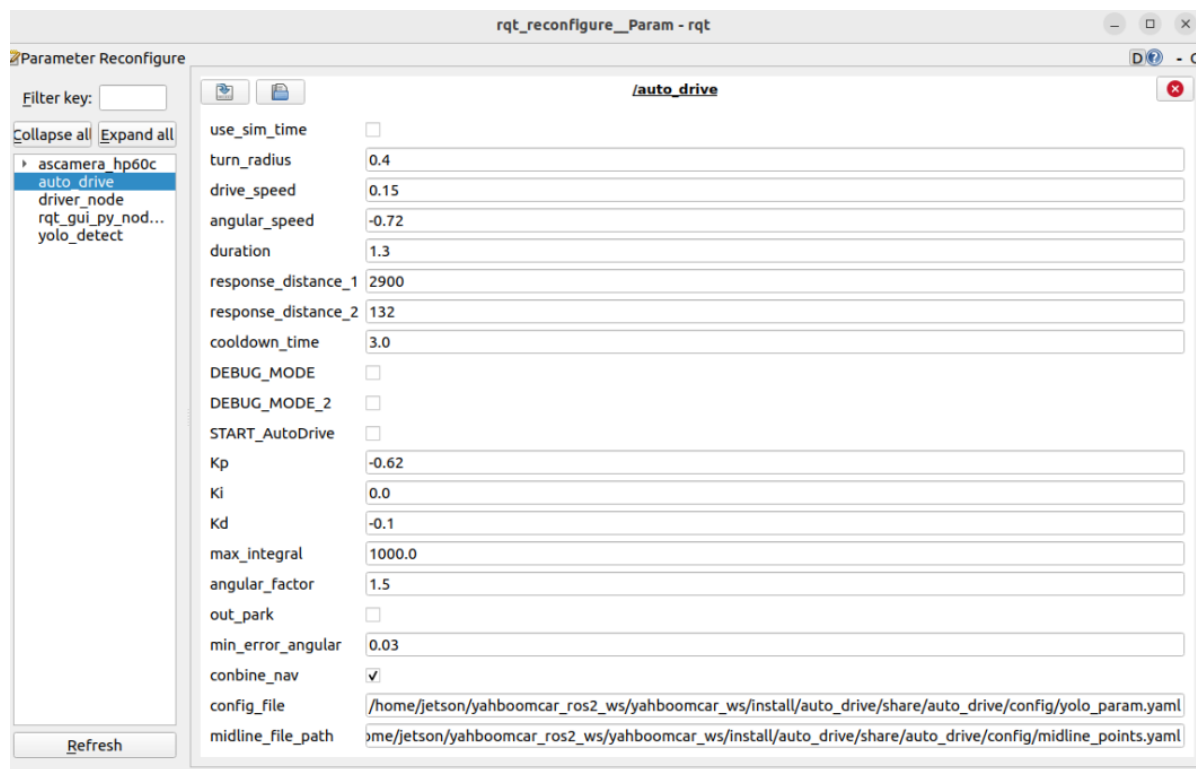
```
jetson@yahboom: ~  
jetson@yahboom: ~ 80x24  
[auto_drive-4] Hello from the pygame community. https://www.pygame.org/contribut  
e.html  
[auto_drive-4] standardized_midline_points start_point:(360, 477) end_point:(348  
, 357),midline_length:120  
[auto_drive-4] Loading /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_dr  
ive/tools/lane_V6.engine for TensorRT inference...  
[ascamera_node-1] [INFO] [1764669566.285700440] [ascamera_hp60c.camera_publisher  
]: 2025-12-02 17:59:26[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gain  
ret 0, gain 4  
[yolo_detect-3] [12/02/2025-17:59:26] [TRT] [I] Loaded engine size: 12 MiB  
[auto_drive-4] [12/02/2025-17:59:26] [TRT] [I] Loaded engine size: 11 MiB  
[yolo_detect-3] [12/02/2025-17:59:26] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performa  
nce or even cause errors.  
[auto_drive-4] [12/02/2025-17:59:26] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performan  
ce or even cause errors.  
[auto_drive-4] [12/02/2025-17:59:27] [TRT] [I] [MemUsageChange] TensorRT-managed  
allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28 (M  
iB)  
[yolo_detect-3] [12/02/2025-17:59:27] [TRT] [I] [MemUsageChange] TensorRT-manage  
d allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28  
(MiB)
```

After the program runs, it will display the YOLO recognition screen. The car will run in the center of the lane line and perform corresponding actions based on the road signs it recognizes during the journey. (If the terminal doesn't report any errors after starting the program, but the robot doesn't move, you can run the following program to check the parameters.)



- Start the dynamic parameter configuration node

```
ros2 run rqt_reconfigure rqt_reconfigure
```

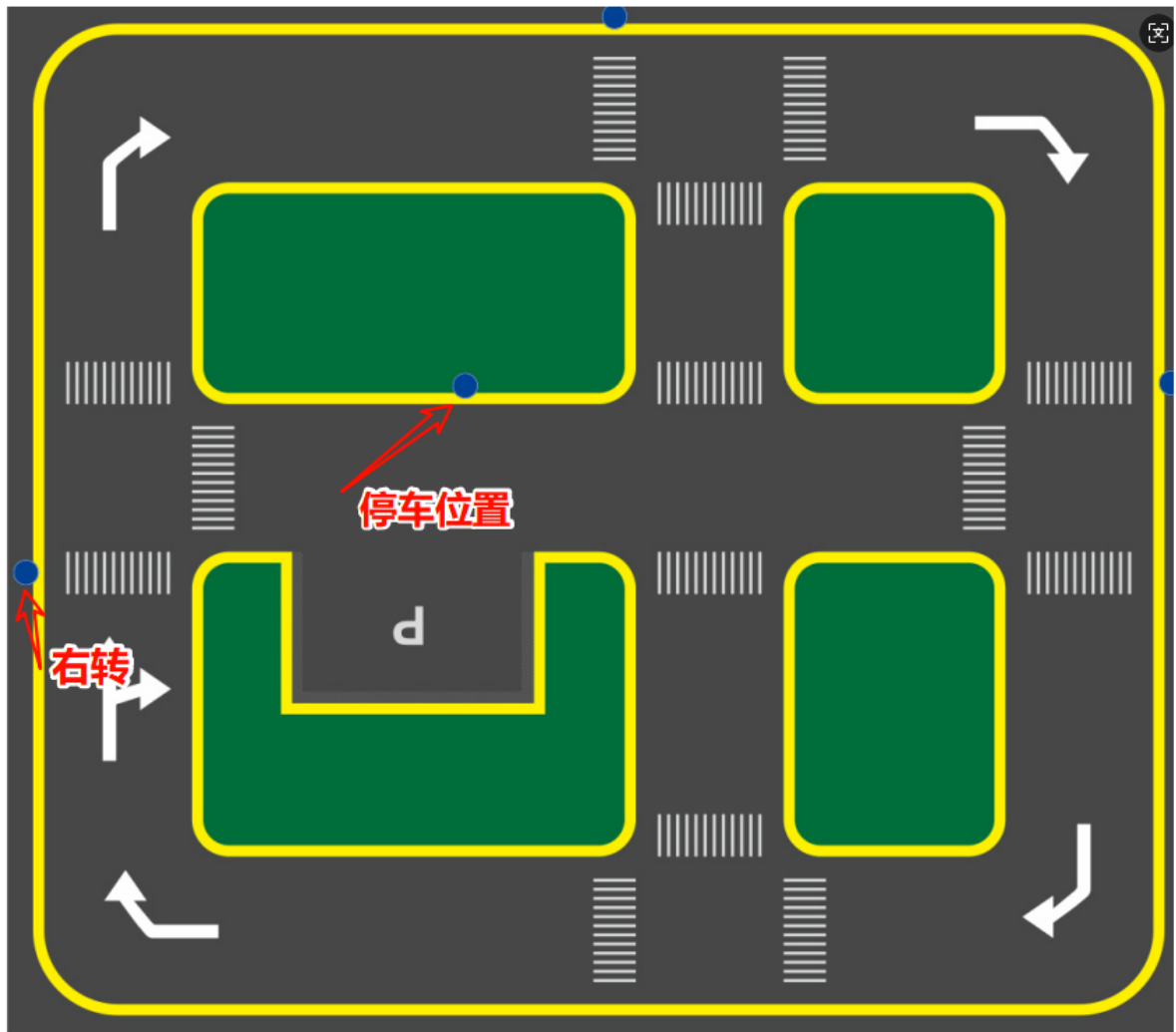


If the robot doesn't move, check two parameters: Is START_AutoDrive checked? Is combine_nav false?

☒ After modifying the parameters, click on the blank space in the GUI to enter the parameter values. Note that this will only take effect for the current startup; for permanent effects, you need to modify the parameters in the source code.

3.2 Recognizing the Car's Movement After a Right Turn

Note: Because the right turn uses a fixed action, the right turn and stop A road signs need to be placed in fixed positions.



The program will print the following when it recognizes a right turn:

```

jetson@yahboom: ~
jetson@yahboom: ~ 80x24
MiB)
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-manage
d allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28
(MiB)
[auto_drive-4] [INFO] [1764671378.387579116] [auto_drive]: current_sign redlight
[auto_drive-4] [INFO] [1764671378.390984163] [auto_drive]: red_stop
[auto_drive-4] [INFO] [1764671832.607130196] [auto_drive]: current_sign greenlig
ht
[auto_drive-4] [INFO] [1764671832.608561506] [auto_drive]: greenlight
[auto_drive-4] [INFO] [1764673308.321102763] [auto_drive]: current_sign parkA
[auto_drive-4] [INFO] [1764673308.323742671] [auto_drive]: parking A
[auto_drive-4] [INFO] [1764674001.122264527] [auto_drive]: current_sign speedlim
it
[auto_drive-4] [INFO] [1764674001.122991778] [auto_drive]: slow down
[auto_drive-4] [INFO] [1764674013.459037728] [auto_drive]: current_sign stop
[auto_drive-4] [INFO] [1764674016.463009335] [auto_drive]: Stop action completed
[auto_drive-4] [INFO] [1764674377.727182525] [auto_drive]: current_sign turnrigh
t
[auto_drive-4] [INFO] [1764674377.728323130] [auto_drive]: Turn action preparing
: right, waiting 2 seconds
[auto_drive-4] [INFO] [1764674377.729364725] [auto_drive]: Turn action started:
right
[auto_drive-4] [INFO] [1764674378.732186552] [auto_drive]: turn right

```

The car will perform a fixed action to make a steering decision. Upon seeing a right turn, it will stop for one second, then proceed for five seconds before turning right.



The steering decision processing function is located in the `auto_drive.py` file. Below is the function that is entered after a right turn is detected. Users can modify it themselves to see how the car moves after recognizing a road sign.

```
def turn(self,direction):
    '''Turn'''
    self.get_logger().info(f'Turn action preparing: {direction}, waiting 2
seconds...')

    self.turn_action_active = True # Set steering action indicator
    self.get_logger().info(f'Turn action started: {direction}')
    self.__set_current_speed(linear_x=0.0, angular_z=0.0 )
    time.sleep(1)

    if self.R == 0:#Turning in place
        if direction == 'left':
            self.get_logger().info('turn left')
            self.__set_current_speed(angular_z=self.angular_speed )
        elif direction == 'right':
            self.get_logger().info('turn right')
            self.__set_current_speed(angular_z=-self.angular_speed )
    else:#Turning with radius
        if direction == 'left':
            self.get_logger().info('turn left')
            self.__set_current_speed(linear_x=self.drive_speed,
angular_z=self.drive_speed / self.R )
        elif direction == 'right':
            self.get_logger().info('turn right')
```

```

        self.__set_current_speed(linear_x=self.drive_speed,
angular_z=-0.03)
        time.sleep(5)
        self.__set_current_speed(linear_x=self.drive_speed,
angular_z=-0.8)
        time.sleep(self.duration)
        self.turn_action_active = False # clear the steering action sign

```

4. Source Code Analysis

4.1 auto_drive.py

YOLO signpost message reception:

```

def sign_callback(self, msg:DetectionObject):
    '''Subscribe to YOLO recognition results'''
    class_name = msg.class_name
    object_distance = self._get_distance(msg.ymin) # Calculate distance
    object = Object(msg.class_name, msg.confidence, object_distance)

```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```

# 1. Check if any other actions are being performed.
if self.action_runing:
    return

# 2. Check cooldown time (to prevent repeated triggering)
current_time = time.time()
if class_name in self.last_processed_time:
    time_diff = current_time - self.last_processed_time[class_name]
    if time_diff < self.cooldown_time: # Default cooldown: 3 seconds
        return

# 3. Check if it is a duplicate road sign.
if class_name == self.last_sign:
    return

# 4. Check if the road sign is enabled.
if not self.sign_enable_list[对应的索引]:
    return

```

Event queue processing

```

def handle_events(self):
    '''Handling target recognition events'''
    while True:
        if not self.event_queue.empty():
            object = self.event_queue.get()
            self.action_runing = True
            current_sign = object.class_name

            try:
                # Perform the appropriate action based on the type of road sign.

```

```

        if object.class_name == "straight" and
self.sign_enable_list[10]:
            self.go_straight()
        elif object.class_name == "turnright" and
self.sign_enable_list[11]:
            self.turn('right')
            # ... Other road sign processing

    finally:
        # Update status record
        self.last_processed_time[current_sign] = time.time()
        self.last_sign = current_sign
        self.action_runing = False

```

Specific action implementation

```

def turn(self,direction):
    '''Turn'''
    self.get_logger().info(f'Turn action preparing: {direction}, waiting 2
seconds...')

    self.turn_action_active = True # Set steering action indicator
    self.get_logger().info(f'Turn action started: {direction}')
    self.__set_current_speed(linear_x=0.0, angular_z=0.0 )
    time.sleep(1)

    if self.R == 0:#Turning in place
        if direction == 'left':
            self.get_logger().info('turn left')
            self.__set_current_speed(angular_z=self.angular_speed )
        elif direction == 'right':
            self.get_logger().info('turn right')
            self.__set_current_speed(angular_z=-self.angular_speed )
    else:#Turning with radius
        if direction == 'left':
            self.get_logger().info('turn left')
            self.__set_current_speed(linear_x=self.drive_speed,
angular_z=self.drive_speed / self.R )
        elif direction == 'right':
            self.get_logger().info('turn right')
            self.__set_current_speed(linear_x=self.drive_speed,
angular_z=-0.03)
        time.sleep(5)
        self.__set_current_speed(linear_x=self.drive_speed,
angular_z=-0.8)
        time.sleep(self.duration)
    self.turn_action_active = False # Clear the steering action sign

```

